on, remembering to always modulo the resulting index into the valid range of the table.

When using closed hashing, it is a good idea to make your table size a *prime number*. Using a prime table size in conjunction with quadratic probing tends to yield the best coverage of the available table slots with minimal clustering. See http://stackoverflow.com/questions/1145217/why-should-hash-functions-use-a-prime-number-modulus for a good discussion of why prime hash table sizes are preferable.

6.3.6.4 Robin Hood Hashing

Robin Hood hashing is another probing method for closed hash tables that has gained popularity recently. This probing scheme improves the performance of a closed hash table, even when the table is nearly full. For a good discussion of how Robin Hood hashing works, see https://www.sebastiansylvan.com/post/robin-hood-hashing-should-be-your-default-hash-table-implementation/.

6.4 Strings

Strings are ubiquitous in almost every software project, and game engines are no exception. On the surface, the string may seem like a simple, fundamental data type. But, when you start using strings in your projects, you will quickly discover a wide range of design issues and constraints, all of which must be carefully accounted for.

6.4.1 The Problem with Strings

The most fundamental question about strings is how they should be stored and managed in your program. In C and C++, strings aren't even an atomic type—they are implemented as *arrays of characters*. The variable length of strings means we either have to hard-code limitations on the sizes of our strings, or we need to dynamically allocate our string buffers.

Another big string-related problem is that of *localization*—the process of adapting your software for release in other languages. This is also known as *internationalization*, or I18N for short. Any string that you display to the user in English must be translated into whatever languages you plan to support. (Strings that are used internally to the program but are never displayed to the user are exempt from localization, of course.) This not only involves making sure that you can represent all the character glyphs of all the languages you plan to support (via an appropriate set of fonts), but it also means ensuring that your game can handle different text orientations. For example, traditional

Chinese text is oriented vertically instead of horizontally (although modern Chinese and Japanese are commonly written horizontally and left-to-right), and some languages like Hebrew read right-to-left. Your game also needs to gracefully deal with the possibility that a translated string will be either much longer or much shorter than its English counterpart.

Finally, it's important to realize that strings are used internally within a game engine for things like resource file names and object ids. For example, when a game designer lays out a level, it's highly convenient to permit him or her to identify the objects in the level using meaningful names, like "PlayerCamera," "enemy-tank-01" or "explosionTrigger."

How our engine deals with these internal strings often has pervasive ramifications on the performance of the game. This is because strings are inherently expensive to work with at runtime. Comparing or copying `int`s or `float`s can be accomplished via simple machine language instructions. On the other hand, comparing strings requires an $O(n)$ scan of the character arrays using a function like `strcmp()` (where n is the length of the string). Copying a string requires an $O(n)$ memory copy, not to mention the possibility of having to dynamically allocate the memory for the copy. During one project I worked on, we profiled our game's performance only to discover that `strcmp()` and `strcpy()` were the top two most expensive functions! By eliminating unnecessary string operations and using some of the techniques outlined in this section, we were able to all but eliminate these functions from our profile and increase the game's frame rate significantly. (I've heard similar stories from developers at a number of different studios.)

6.4.2 String Classes

Many C++ programmers prefer to use a *string class*, such as the C++ standard library's `std::string`, rather than deal directly with character arrays. Such classes can make working with strings much more convenient for the programmer. However, a string class can have hidden costs that are difficult to see until the game is profiled. For example, passing a string to a function using a C-style character array is fast because the address of the first character is typically passed in a hardware register. On the other hand, passing a string object might incur the overhead of one or more copy constructors, if the function is not declared or used properly. Copying strings might involve dynamic memory allocation, causing what looks like an innocuous function call to end up costing literally thousands of machine cycles.

Because of the myriad issues with string classes, I generally prefer to avoid them in runtime game code. However, if you feel a strong urge to use a string

class, make sure you pick or implement one that has acceptable runtime performance characteristics—and be sure all programmers that use it are aware of its costs. Know your string class: Does it treat all string buffers as read-only? Does it utilize the *copy on write* optimization? (See http://en.wikipedia.org/wiki/Copy-on-write.) In C++11, does it provide a move constructor? Does it own the memory associated with the string, or can it reference memory that it does not own? (See http://www.boost.org/doc/libs/1_57_0/libs/utility/doc/html/string_ref.html for more on the issue of memory ownership in string classes.) As a rule of thumb, always pass string objects by reference, never by value (as the latter often incurs string-copying costs). Profile your code early and often to ensure that your string class isn't becoming a major source of lost frame rate!

One situation in which a specialized string class does seem justifiable to me is when storing and managing file system paths. Here, a hypothetical `Path` class could add significant value over a raw C-style character array. For example, it might provide functions for extracting the filename, file extension or directory from the path. It might hide operating system differences by automatically converting Windows-style backslashes to UNIX-style forward slashes or some other operating system's path separator. Writing a `Path` class that provides this kind of functionality in a cross-platform way could be highly valuable within a game engine context. (See Section 7.1.1.4 for more details on this topic.)

6.4.3 Unique Identifiers

The *objects* in any virtual game world need to be uniquely identified in some way. For example, in Pac Man we might encounter game objects named "pac_man," "blinky," "pinky," "inky" and "clyde." Unique object identifiers allow game designers to keep track of the myriad objects that make up their game worlds and also permit those objects to be found and operated on at runtime by the engine. In addition, the *assets* from which our game objects are constructed—meshes, materials, textures, audio clips, animations and so on—all need unique identifiers as well.

Strings seem like a natural choice for such identifiers. Assets are often stored in individual files on disk, so they can usually be identified uniquely by their file paths, which of course are strings. And game objects are created by game designers, so it is natural for them to assign their objects understandable string names, rather than have to remember integer object indices, or 64- or 128-bit globally unique identifiers (GUIDs). However, the speed with which *comparisons* between unique identifiers can be made is of paramount impor-

tance in a game, so `strcmp()` simply doesn't cut it. We need a way to have our cake and eat it too—a way to get all the descriptiveness and flexibility of a string, but with the speed of an integer.

6.4.3.1 Hashed String Ids

One good solution is to *hash* our strings. As we've seen, a hash function maps a string onto a semi-unique integer. String hash codes can be compared just like any other integers, so comparisons are fast. If we store the actual strings in a hash table, then the original string can always be recovered from the hash code. This is useful for debugging purposes and to permit hashed strings to be displayed on-screen or in log files. Game programmers sometimes use the term *string id* to refer to such a hashed string. The Unreal engine uses the term *name* instead (implemented by class `FName`).

As with any hashing system, *collisions* are a possibility (i.e., two different strings might end up with the same hash code). However, with a suitable hash function, we can all but guarantee that collisions will not occur for all reasonable input strings we might use in our game. After all, a 32-bit hash code represents more than four billion possible values. So, if our hash function does a good job of distributing strings evenly throughout this very large range, we are unlikely to collide. At Naughty Dog, we started out using a variant of the CRC-32 algorithm to hash our strings, and we encountered only a handful of collisions during many years of development on *Uncharted* and *The Last of Us*. And when a collision did occur, fixing it was a simple matter of slightly altering one of the strings (e.g., append a "2" or a "b" to one of the strings, or use a totally different but synonymous string). That being said, Naughty Dog has moved to a 64-bit hashing function for *The Last of Us Part II* and all of our future game titles; this should essentially eliminate the possibility of hash collisions, given the quantity and typical lengths of the strings we use in any one game.

6.4.3.2 Some Implementation Ideas

Conceptually, it's easy enough to run a hash function on your strings in order to generate string ids. Practically speaking, however, it's important to consider *when* the hash will be calculated. Most game engines that use string ids do the hashing at runtime. At Naughty Dog, we permit runtime hashing of strings, but we also use C++11's *user-defined literals* feature to transform the syntax `"any_string"_sid` directly into a hashed integer value at compile time. This permits string ids to be used anywhere that an integer manifest constant can be used, including the constant `case` labels of a `switch` statement. (The result of a function call that generates a string id at runtime is not

a constant, so it cannot be used as a `case` label.)

The process of generating a string id from a string is sometimes called *interning* the string, because in addition to hashing it, the string is typically also added to a global string table. This allows the original string to be recovered from the hash code later. You may also want your tools to be capable of hashing strings into string ids. That way, when the tool generates data for consumption by your engine, the strings will already have been hashed.

The main problem with interning a string is that it is a slow operation. The hashing function must be run on the string, which can be an expensive proposition, especially when a large number of strings are being interned. In addition, memory must be allocated for the string, and it must be copied into the lookup table. As a result (if you are not generating string ids at compile-time), it is usually best to intern each string only once and save off the result for later use. For example, it would be preferable to write code like this because the latter implementation causes the strings to be unnecessarily re-interned every time the function `f()` is called.

```
static StringId   sid_foo = internString("foo");
static StringId   sid_bar = internString("bar");

// ...

void f(StringId id)
{
    if (id == sid_foo)
    {
        // handle case of id == "foo"
    }
    else if (id == sid_bar)
    {
        // handle case of id == "bar"
    }
}
```

The following approach is less efficient:

```
void f(StringId id)
{
    if (id == internString("foo"))
    {
        // handle case of id == "foo"
    }
```

```
    else if (id == internString("bar"))
    {
        // handle case of id == "bar"
    }
}
```

Here's one possible implementation of `internString()`.

stringid.h

```
typedef U32 StringId;

extern StringId internString(const char* str);
```

stringid.cpp

```
static HashTable<StringId, const char*> gStringIdTable;

StringId internString(const char* str)
{
    StringId sid = hashCrc32(str);

    HashTable<StringId, const char*>::iterator it
        = gStringIdTable.find(sid);

    if (it == gStringTable.end())
    {
        // This string has not yet been added to the
        // table. Add it, being sure to copy it in case
        // the original was dynamically allocated and
        // might later be freed.
        gStringTable[sid] = strdup(str);
    }

    return sid;
}
```

Another idea employed by the Unreal Engine is to wrap the string id and a pointer to the corresponding C-style character array in a tiny class. In the Unreal Engine, this class is called `FName`. At Naughty Dog we do the same and wrap our string ids in a `StringId` class. We define a macro so that `SID("any_string")` produces an instance of this class, with its hashed value produced by our user-defined string literal syntax `"any_string"_sid`.

Using Debug Memory for Strings

When using string ids, the strings themselves are only kept around for human consumption. When you ship your game, you almost certainly won't need the strings—the game itself should only ever use the ids. As such, it's a good idea to store your string table in a region of memory that won't exist in the retail game. For example, a PS3 development kit has 256 MiB of retail memory, plus an additional 256 MiB of "debug" memory that is not available on a retail unit. If we store our strings in debug memory, we needn't worry about their impact on the memory footprint of the final shipping game. (We just need to be careful never to write production code that depends on the strings being available!)

6.4.4 Localization

Localization of a game (or any software project) is a big undertaking. It is a task best handled by planning for it from day one and accounting for it at every step of development. However, this is not done as often as we all would like. Here are some tips that should help you plan your game engine project for localization. For an in-depth treatment of software localization, see [34].

6.4.4.1 Unicode

The problem for most English-speaking software developers is that they are trained from birth (or thereabouts!) to think of strings as arrays of eight-bit ASCII character codes (i.e., characters following the ANSI standard). ANSI strings work great for a language with a simple alphabet, like English. But, they just don't cut it for languages with complex alphabets containing a great many more characters, sometimes totally different glyphs than English's 26 letters. To address the limitations of the ANSI standard, the Unicode character set system was devised.

The basic idea behind Unicode is to assign every character or glyph from every language in common use around the globe to a unique hexadecimal code known as a *code point*. When storing a string of characters in memory, we select a particular *encoding*—a specific means of representing the Unicode code points for each character—and following those rules, we lay down a sequence of bits in memory that represent the string. UTF-8 and UTF-16 are two common encodings. You should select the specific encoding standard that best suits your needs.

Please set down this book right now and read the article entitled, "The Absolute Minimum Every Software Developer Absolutely, Positively Must Know About Unicode and Character Sets (No Excuses!)" by Joel Spolsky. You can find it here: http://www.joelonsoftware.com/articles/Unicode.html. (Once you've done that, please pick up the book again!)

UTF-32

The simplest Unicode encoding is UTF-32. In this encoding, each Unicode code point is encoded into a 32-bit (4-byte) value. This encoding wastes a lot of space, for two reasons: First, most strings in Western European languages do not use any of the highest-valued code points, so an average of at least 16 bits (2 bytes) is usually wasted per character. Second, the highest Unicode code point is 0x10FFFF, so even if we wanted to create a string that uses every possible Unicode glyph, we'd still only need 21 bits per character, not 32.

That said, UTF-32 does have simplicity in its favor. It is a *fixed-length* encoding, meaning that every character occupies the same number of bits in memory (32 bits to be precise). As such, we can determine the length of any UTF-32 string by taking its length in bytes and dividing by four.

UTF-8

In the UTF-8 encoding scheme, the code points for each character in a string are stored using eight-bit (one-byte) granularity, but some code points occupy more than one byte. Hence the number of bytes occupied by a UTF-8 character string is not necessarily the length of the string in characters. This is known as a *variable-length* encoding, or a *multibyte character set* (MBCS), because each character in a string may take one or more bytes of storage.

One of the big benefits of the UTF-8 encoding is that it is backwards-compatible with the ANSI encoding. This works because the first 127 Unicode code points correspond numerically to the old ANSI character codes. This means that every ANSI character will be represented by exactly one byte in UTF-8, and a string of ANSI characters can be interpreted as a UTF-8 string without modification.

To represent higher-valued code points, the UTF-8 standard uses multibyte characters. Each multibyte character starts with a byte whose most-significant bit is 1 (i.e., its value lies in the range 128–255, inclusive). Such high-valued bytes will never appear in an ANSI string, so there is no ambiguity when distinguishing between single-byte characters and multibyte characters.

UTF-16

The UTF-16 encoding employs a somewhat simpler, albeit more expensive approach. Each character in a UTF-16 string is represented by either one or two 16-bit values. The UTF-16 encoding is known as a wide character set (WCS) because each character is at least 16 bits wide, instead of the eight bits used by

"regular" ANSI `char`s and their UTF-8 counterparts.

In UTF-16, the set of all possible Unicode code points is divided into 17 *planes* containing 2^{16} code points each. The first plane is known as the *basic multilingual plane* (BMP). It contains the most commonly used code points across a wide range of languages. As such, many UTF-16 strings can be represented entirely by code points within the first plane, meaning that each character in such a string is represented by only *one* 16-bit value. However, if a character from one of the other planes (known as *supplementary planes*) is required in a string, it is represented by two consecutive 16-bit values.

The UCS-2 (2-byte universal character set) encoding is a limited subset of the UTF-16 encoding, utilizing *only* the basic multilingual page. As such, it cannot represent characters whose Unicode code points are numerically higher than 0xFFFF. This simplifies the format, because every character is guaranteed to occupy exactly 16 bits (two bytes). In other words, UCS-2 is a *fixed-length* character encoding, while in general UTF-8 and UTF-16 are *variable-length* encodings.

If we know a priori that a UTF-16 string only utilizes code points from the BMP (or if we are dealing with a UCS-2 encoded string), we can determine the number of characters in the string by simply dividing the number of bytes by two. Of course, if supplemental planes are used in a UTF-16 string, this simple "trick" no longer works.

Note that a UTF-16 encoding can be little-endian or big-endian (see Section 3.3.2.1), depending on the native endianness of your target CPU. When storing UTF-16 text on-disc, it's common to precede the text data with a *byte order mark* (BOM) indicating whether the individual 16-bit characters are stored in little- or big-endian format. (This is true of UTF-32 encoded string data as well, of course.)

6.4.4.2 `char` versus `wchar_t`

The standard C/C++ library defines two data types for dealing with character strings—`char` and `wchar_t`. The `char` type is intended for use with legacy ANSI strings and with multibyte character sets (MBCS), including (but not limited to) UTF-8. The `wchar_t` type is a "wide" character type, intended to be capable of representing any valid code point in a single integer. As such, its size is compiler- and system-specific. It could be eight bits on a system that does not support Unicode at all. It could be 16 bits if the UCS-2 encoding is assumed for all wide characters, or if a multi-word encoding like UTF-16 is being employed. Or it could be 32 bits if UTF-32 is the "wide" character encoding of choice.

Because of this inherent ambiguity in the definition of `wchar_t`, if you

need to write truly portable string-handling code, you'll need to define your own character data type(s) and provide a library of functions for dealing with whatever Unicode encoding(s) you need to support. However, if you are targeting a specific platform and compiler, you can write your code within the limits of that particular implementation, at the loss of some portability.

The following article does a good job of outlining the pros and cons of using the `wchar_t` data type: http://icu-project.org/docs/papers/unicode_wchar_t.html.

6.4.4.3 Unicode under Windows

Under Windows, the `wchar_t` data type is used exclusively for UTF-16 encoded Unicode strings, and the `char` type is used for ANSI strings and legacy *Windows code page* string encodings. When reading the Windows API docs, the term "Unicode" is therefore always synonymous with "wide character set" (WCS) and UTF-16 encoding. This is a bit confusing, because of course Unicode strings can in general be encoded in the "non-wide" multibyte UTF-8 format.

The Windows API defines three sets of character/string manipulation functions: one set for single-byte character set ANSI strings (SBCS), one set for multibyte character set (MBCS) strings, and one set for wide character set strings. The ANSI functions are essentially the old-school "C-style" string functions we all grew up with. The MBCS string functions handle a variety of multibyte encodings and are primarily designed for dealing with legacy Windows code pages encodings. The WCS functions handle Unicode UTF-16 strings.

Throughout the Windows API, a prefix or suffix of "w," "wcs" or "W" indicates a wide character set (UTF-16) encoding; a prefix or suffix of "mb" indicates a multibyte encoding; and a prefix or suffix of "a" or "A," or the lack of any prefix or suffix, indicates an ANSI or Windows code pages encoding. The C++ standard library uses a similar convention—for example, `std::string` is its ANSI string class, while `std::wstring` is its wide character equivalent. Unfortunately, the names of the functions aren't always 100% consistent. This all leads to some confusion among programmers who aren't in the know. (But *you* aren't one of those programmers!) Table 6.2 lists some examples.

Windows also provides functions for translating between ANSI character strings, multibyte strings and wide UTF-16 strings. For example, `wcstombs()` converts a wide UTF-16 string into a multibyte string according to the currently active *locale* setting.

The Windows API uses a little preprocessor trick to allow you to write code that is at least superficially portable between wide (Unicode) and non-wide

ANSI	WCS	MBCS
`strcmp()`	`wcscmp()`	`_mbscmp()`
`strcpy()`	`wcscpy()`	`_mbscpy()`
`strlen()`	`wcslen()`	`_mbstrlen()`

Table 6.2. Variants of some common C standard library string functions for use with ANSI, wide and multibyte character sets.

(ANSI/MBCS) string encodings. The generic character data type `TCHAR` is defined to be a `typedef` to `char` when building your application in "ANSI mode," and it's defined to be a `typedef` to `wchar_t` when building your application in "Unicode mode." The macro `_T()` is used to convert an eight-bit string literal (e.g., `char* s = "this is a string";`) into a wide string literal (e.g., `wchar_t* s = L"this is a string";`) when compiling in "Unicode mode." Likewise, a suite of "fake" API functions are provided that "automagically" morph into their appropriate 8-bit or 16-bit variant, depending on whether you are building in "Unicode mode" or not. These magic character-set-independent functions are either named with no prefix or suffix, or with a "t," "tcs" or "T" prefix or suffix.

Complete documentation for all of these functions can be found on Microsoft's MSDN website. Here's a link to the documentation for `strcmp()` and its ilk, from which you can quite easily navigate to the other related string-manipulation functions using the tree view on the left-hand side of the page, or via the search bar: http://msdn2.microsoft.com/en-us/library/kk6xf663(VS.80).aspx.

6.4.4.4 Unicode on Consoles

The Xbox 360 software development kit (XDK) uses WCS strings pretty much exclusively, for all strings—even for internal strings like file paths. This is certainly one valid approach to the localization problem, and it makes for very consistent string handling throughout the XDK. However, the UTF-16 encoding is a bit wasteful on memory, so different game engines may employ different conventions. At Naughty Dog, we use eight-bit `char` strings throughout our engine, and we handle foreign languages via a UTF-8 encoding. The choice of encoding is not particularly important, as long as you select one as early in the project as possible and stick with it consistently.

6.4.4.5 Other Localization Concerns

Even once you have adapted your software to use Unicode characters, there is still a host of other localization problems to contend with. For one thing,

Id	English	French
p1score	“Player 1 Score”	“Joueur 1 Score”
p2score	“Player 2 Score”	“Joueur 2 Score”
p1wins	“Player one wins!”	“Joueur un gagne!”
p2wins	“Player two wins!”	“Joueur deux gagne!”

Table 6.3. Example of a string database used for localization.

strings aren’t the only place where localization issues arise. Audio clips including recorded voices must be translated. Textures may have English words painted into them that require translation. Many symbols have different meanings in different cultures. Even something as innocuous as a no-smoking sign might be misinterpreted in another culture. In addition, some markets draw the boundaries between the various game-rating levels differently. For example, in Japan a Teen-rated game is not permitted to show blood of any kind, whereas in North America small red blood spatters are considered acceptable.

For strings, there are other details to worry about as well. You will need to manage a database of all human-readable strings in your game, so that they can all be reliably translated. The software must display the proper language given the user’s installation settings. The formatting of the strings may be totally different in different languages—for example, Chinese is sometimes written vertically, and Hebrew reads right-to-left. The lengths of the strings will vary greatly from language to language. You’ll also need to decide whether to ship a single DVD or Blu-ray disc that contains all languages or ship different discs for particular territories.

The most crucial components in your localization system will be the central database of human-readable strings and an in-game system for looking up those strings by id. For example, let’s say you want a heads-up display that lists the score of each player with “Player 1 Score:” and “Player 2 Score:” labels and that also displays the text “Player 1 Wins” or “Player 2 Wins” at the end of a round. These four strings would be stored in the localization database under unique ids that are understandable to you, the developer of the game. So our database might use the ids “p1score,” “p2score,” “p1wins” and “p2wins,” respectively. Once our game’s strings have been translated into French, our database would look something like the simple example shown in Table 6.3. Additional columns can be added for each new language your game supports.

The exact format of this database is up to you. It might be as simple as a Microsoft Excel worksheet that can be saved as a comma-separated values (CSV) file and parsed by the game engine or as complex as a full-fledged Or-

acle database. The specifics of the string database are largely unimportant to the game engine, as long as it can read in the string ids and the corresponding Unicode strings for whatever language(s) your game supports. (However, the specifics of the database may be *very* important from a practical point of view, depending upon the organizational structure of your game studio. A small studio with in-house translators can probably get away with an Excel spreadsheet located on a network drive. But a large studio with branch offices in Britain, Europe, South America and Japan would probably find some kind of distributed database a great deal more amenable.)

At runtime, you'll need to provide a simple function that returns the Unicode string in the "current" language, given the unique id of that string. The function might be declared like this:

```
wchar_t getLocalizedString(const char* id);
```

and it might be used like this:

```
void drawScoreHud(const Vector3& score1Pos,
                  const Vector3& score2Pos)
{
    renderer.displayTextOrtho(getLocalizedString("p1score"),
                              score1Pos);

    renderer.displayTextOrtho(getLocalizedString("p2score"),
                              score2Pos);

    // ...
}
```

Of course, you'll need some way to set the "current" language globally. This might be done via a configuration setting, which is fixed during the installation of the game. Or you might allow users to change the current language on the fly via an in-game menu. Either way, the setting is not difficult to implement; it can be as simple as a global integer variable specifying the index of the column in the string table from which to read (e.g., column one might be English, column two French, column three Spanish and so on).

Once you have this infrastructure in place, your programmers must remember to *never display a raw string to the user*. They must always use the id of a string in the database and call the look-up function in order to retrieve the string in question.

6.4.4.6 Case Study: Naughty Dog's Localization Tool

At Naughty Dog, we use a localization database that we developed in-house. The localization tool's back end consists of a MySQL database located on a

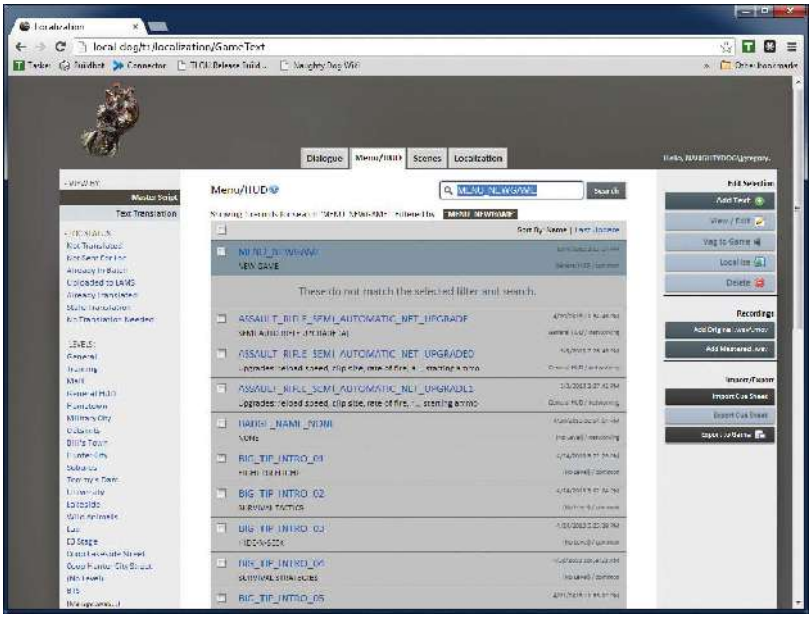

Figure 6.10. Naughty Dog's localization tool's main window, showing a list of pure text assets used in the menus and HUD. The user has just performed a search for an asset called `MENU_NEWGAME`.

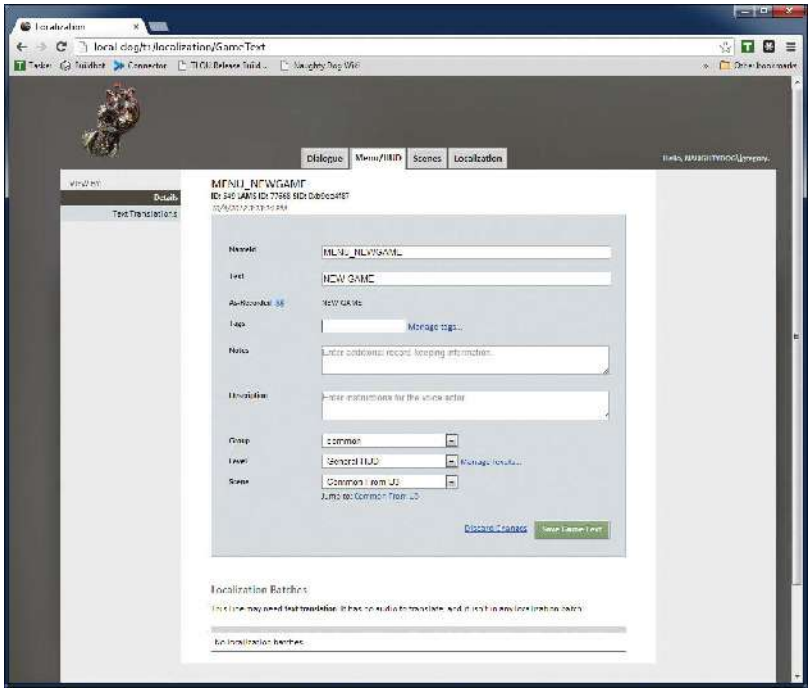

Figure 6.11. Detailed asset view, showing the `MENU_NEWGAME` string.

server that is accessible both to the developers within Naughty Dog and also to the various external companies with which we work to translate our text and speech audio clips into the various languages our games support. The front end is a web interface that "speaks" to the database, allowing users to view all of the text and audio assets, edit their contents, provide translations for each asset, search for assets by id or by content and so on.

In Naughty Dog's localization tool, each asset is either a string (for use in the menus or HUD) or a speech audio clip with optional subtitle text (for use as in-game dialog or within cinematics). Each asset has a unique identifier, which is represented as a hashed string id (see Section 6.4.3.1). If a string is required for use in the menus or HUD, we look it up by its id and get back a Unicode (UTF-8) string suitable for display on-screen. If a line of dialog must be played, we likewise look up the audio clip by its id and use the data in-engine to look up its corresponding subtitle (if any). The subtitle is treated just like a menu or HUD string, in that it is returned by the localization tool's API as a UTF-8 string suitable for display.

Figure 6.10 shows the main interface of the localization tool, in this case displayed in the Chrome web browser. In this image, you can see that the user has typed in the id `MENU_NEWGAME` in order to look up the string "NEW GAME" (used on the game's main menu for launching a new game). Figure 6.11 shows the detailed view of the `MENU_NEWGAME` asset. If the user hits the "Text Translations" button in the upper-left corner of the asset details window, the screen shown in Figure 6.12 comes up, allowing the user to enter or edit the various translations of the string. Figure 6.13 shows another tab on the localization tool's main page, this time listing audio speech assets. Finally, Figure 6.14 depicts the detailed asset view for the speech asset `BADA_GAM_MIL_ESCAPE_OVERPASS_001` ("We missed all the action"), showing translations of this line of dialog into some of the supported languages.

6.5 Engine Configuration

Game engines are complex beasts, and they invariably end up having a large number of configurable options. Some of these options are exposed to the player via one or more options menus in-game. For example, a game might expose options related to graphics quality, the volume of music and sound effects, or controller configuration. Other options are created for the benefit of the game development team only and are either hidden or stripped out of the game completely before it ships. For example, the player character's

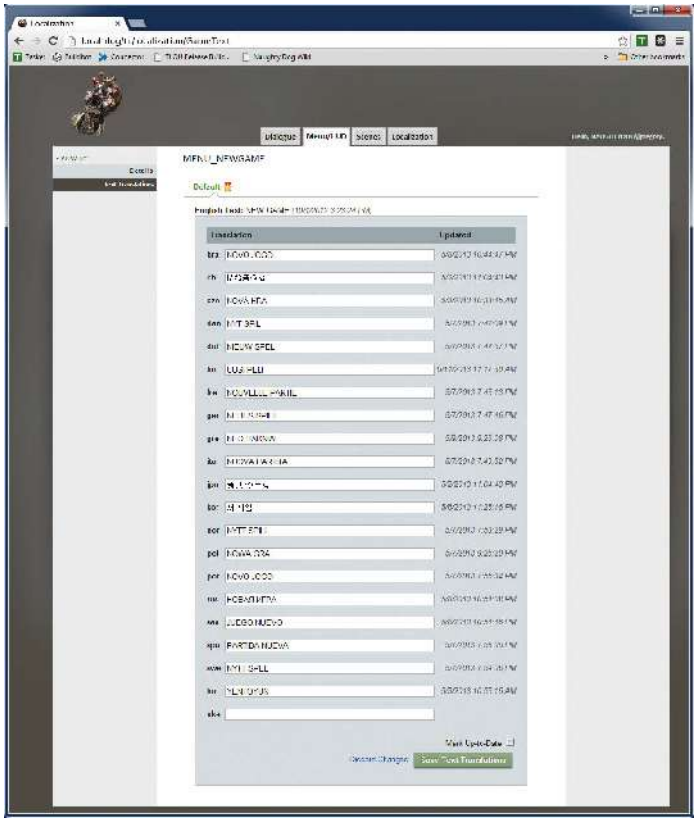

Figure 6.12. Text translations of the string "NEW GAME" into all languages supported by Naughty Dog's *The Last of Us*.

maximum walk speed might be exposed as an option so that it can be fine-tuned during development, but it might be changed to a hard-coded value prior to ship.

6.5.1 Loading and Saving Options

A configurable option can be implemented trivially as a global variable or a member variable of a singleton class. However, configurable options are not particularly useful unless their values can be configured, stored on a hard disk, memory card or other storage medium, and later retrieved by the game. There are a number of simple ways to load and save configuration options:

- *Text configuration files*. By far the most common method of saving and loading configuration options is by placing them into one or more text

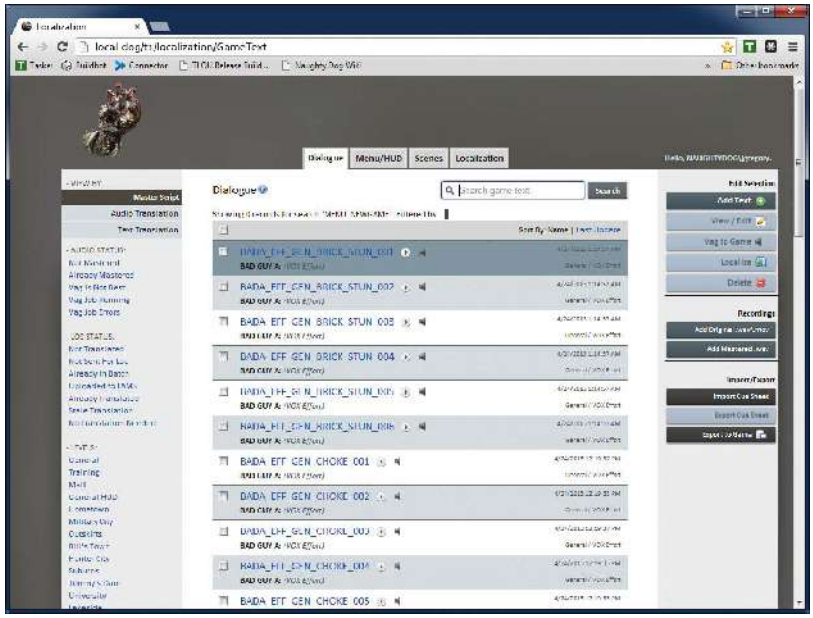

Figure 6.13. Naughty Dog's localization tool's main window again, this time showing a list of speech audio assets with accompanying subtitle text.

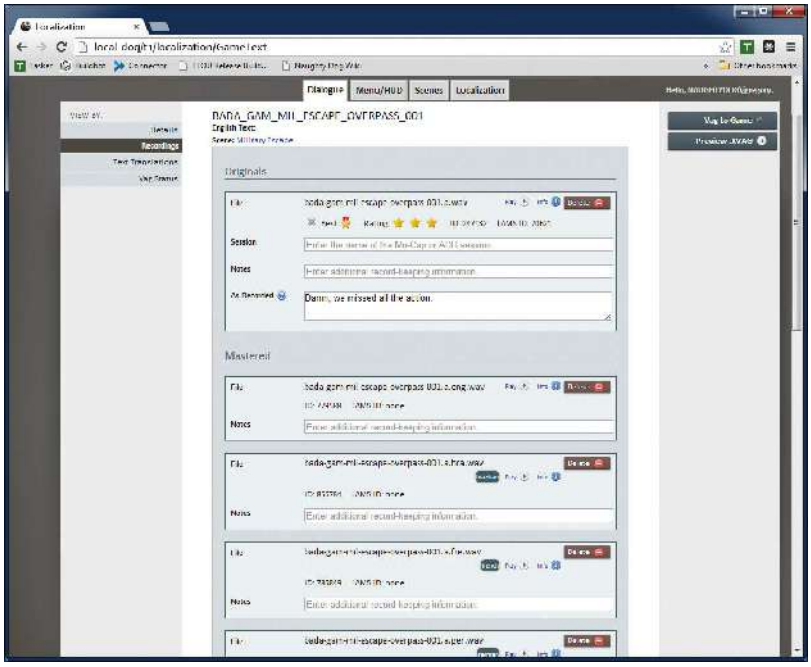

Figure 6.14. Detailed asset view showing recorded translations for the speech asset `BADA_GAM_MIL_ESCAPE_OVERPASS_001` ("We missed all the action").

files. The format of these files varies widely from engine to engine, but it is usually very simple. For example, Windows INI files (which are used by the OGRE renderer) consist of flat lists of key-value pairs grouped into logical sections. The JSON format is another common choice for configurable game options files. XML is another viable option, although most developers these days find JSON to be less verbose and easier to read than XML.

- *Compressed binary files*. Most modern consoles have hard disk drives in them, but older consoles could not afford this luxury. As a result, all game consoles since the Super Nintendo Entertainment System (SNES) have come equipped with proprietary removable memory cards that permit both reading and writing of data. Game options are sometimes stored on these cards, along with saved games. Compressed binary files are the format of choice on a memory card, because the storage space available on these cards is often very limited.

- *The Windows registry*. The Microsoft Windows operating system provides a global options database known as the registry. It is stored as a tree, where the interior nodes (known as *registry keys*) act like file folders, and the leaf nodes store the individual options as key-value pairs. That being said, I don't recommend using the Windows registry for storing engine configuration information. The registry is a monolithic database that can easily be corrupted, lost (when Windows is reinstalled), or thrown out-of-sync with the files in the filesystem. For more on the weaknesses of the Windows registry, see https://blog.codinghorror.com/was-the-windows-registry-a-good-idea/.

- *Command line options*. The command line can be scanned for option settings. The engine might provide a mechanism for controlling any option in the game via the command line, or it might expose only a small subset of the game's options here.

- *Environment variables*. On personal computers running Windows, Linux or MacOS, environment variables are sometimes used to store configuration options as well.

- *Online user profiles*. With the advent of online gaming communities like Xbox Live, each user can create a profile and use it to save achievements, purchased and unlockable game features, game options and other information. The data are stored on a central server and can be accessed by the player wherever an Internet connection is available.

6.5.2 Per-User Options

Most game engines differentiate between global options and per-user options. This is necessary because most games allow each player to configure the game to his or her liking. It is also a useful concept during development of the game, because it allows each programmer, artist and designer to customize his or her work environment without affecting other team members.

Obviously care must be taken to store per-user options in such a way that each player "sees" only his or her options and not the options of other players on the same computer or console. In a console game, the user is typically allowed to save his or her progress, along with per-user options such as controller preferences, in "slots" on a memory card or hard disk. These slots are usually implemented as files on the media in question.

On a Windows machine, each user has a folder under *C:\Users* containing information such as the user's desktop, his or her My Documents folder, his or her Internet browsing history and temporary files and so on. A hidden subfolder named *AppData* is used to store per-user information on a per-application basis; each application creates a folder under *AppData* and can use it to store whatever per-user information it requires.

Windows games sometimes store per-user configuration data in the registry. The registry is arranged as a tree, and one of the top-level children of the root node, called `HKEY_CURRENT_USER`, stores settings for whichever user happens to be logged on. Every user has his or her own subtree in the registry (stored under the top-level subtree `HKEY_USERS`), and `HKEY_CURRENT_USER` is really just an alias to the current user's subtree. So games and other applications can manage per-user configuration options by simply reading and writing them to keys under the `HKEY_CURRENT_USER` subtree.

6.5.3 Configuration Management in Some Real Engines

In this section, we'll take a brief look at how some real game engines manage their configuration options.

6.5.3.1 Example: Quake's Cvars

The Quake family of engines uses a configuration management system known as *console variables*, or *cvars* for short. A cvar is just a floating-point or string global variable whose value can be inspected and modified from within Quake's in-game console. The values of some cvars can be saved to disk and later reloaded by the engine.

At runtime, cvars are stored in a global linked list. Each cvar is a dynamically allocated instance of `struct cvar_t`, which contains the variable's

name, its value as a string or float, a set of flag bits, and a pointer to the next cvar in the linked list of all cvars. Cvars are accessed by calling `Cvar_Get()`, which creates the variable if it doesn't already exist and modified by calling `Cvar_Set()`. One of the bit flags, `CVAR_ARCHIVE`, controls whether or not the cvar will be saved into a configuration file called *config.cfg*. If this flag is set, the value of the cvar will persist across multiple runs of the game.

6.5.3.2 Example: OGRE

The OGRE rendering engine uses a collection of text files in Windows INI format for its configuration options. By default, the options are stored in three files, each of which is located in the same folder as the executable program:

- *plugins.cfg* contains options specifying which optional engine plug-ins are enabled and where to find them on disk.
- *resources.cfg* contains a *search path* specifying where game assets (a.k.a. media, a.k.a. resources) can be found.
- *ogre.cfg* contains a rich set of options specifying which renderer (DirectX or OpenGL) to use and the preferred video mode, screen size, etc.

Out of the box, OGRE provides no mechanism for storing per-user configuration options. However, the OGRE source code is freely available, so it would be quite easy to change it to search for its configuration files in the user's home directory instead of in the folder containing the executable. The `Ogre::ConfigFile` class makes it easy to write code that reads and writes brand new configuration files as well.

6.5.3.3 Example: The Uncharted and The Last of Us Series

Naughty Dog's engine makes use of a number of configuration mechanisms.

In-Game Menu Settings

The Naughty Dog engine supports a powerful in-game menu system, allowing developers to control global configuration options and invoke commands. The data types of the configurable options must be relatively simple (primarily Boolean, integer and floating-point variables), but this limitation has not prevented the developers at Naughty Dog from creating literally hundreds of useful menu-driven options.

Each configuration option is implemented as a global variable, or a member of a singleton struct or class. When the menu option that controls an option is created, the address of the variable is provided, and the menu item directly controls its value. As an example, the following function creates a

submenu item containing some options for Naughty Dog's *rail vehicles* (simple vehicles that ride on splines which have been used in pretty much every Naughty Dog game, from the "Out of the Frying Pan" jeep chase level in *Uncharted: Drake's Fortune* to the truck convoy / jeep chase sequence in *Uncharted 4*). It defines menu items controlling three global variables: two Booleans and one floating-point value. The items are collected onto a menu, and a special item is returned that will bring up the menu when selected. Presumably the code calling this function adds this item to the parent menu that it is building.

```
DMENU::ItemSubmenu * CreateRailVehicleMenu()
{
    extern bool g_railVehicleDebugDraw2D;
    extern bool g_railVehicleDebugDrawCameraGoals;
    extern float g_railVehicleFlameProbability;

    DMENU::Menu * pMenu
        = new DMENU::Menu("RailVehicle");

    pMenu->PushBackItem(
        new DMENU::ItemBool("Draw 2D Spring Graphs",
            DMENU::ToggleBool,
            &g_railVehicleDebugDraw2D));

    pMenu->PushBackItem(
        new DMENU::ItemBool("Draw Goals (Untracked)",
            DMENU::ToggleBool,
            &g_railVehicleDebugDrawCameraGoals));

    DMENU::ItemFloat * pItemFloat;
    pItemFloat = new DMENU::ItemFloat(
        "FlameProbability",
        DMENU::EditFloat, 5, "%5.2f",
        &g_railVehicleFlameProbability);

    pItemFloat->SetRangeAndStep(0.0f, 1.0f, 0.1f, 0.01f);
    pMenu->PushBackItem(pItemFloat);

    DMENU::ItemSubmenu * pSubmenuItem;
    pSubmenuItem = new DMENU::ItemSubmenu(
        "RailVehicle...", pMenu);

    return pSubmenuItem;
}
```

The value of any option can be saved by simply marking it with the circle button on the Dualshock joypad when the corresponding menu item is se-

lected. The menu settings are saved in an INI-style text file, allowing the saved global variables to retain the values across multiple runs of the game. The ability to control which options are saved on a *per-menu-item basis* is highly useful, because any option that is not saved will take on its programmer-specified default value. If a programmer changes a default, all users will "see" the new value, unless of course a user has saved a custom value for that particular option.

Command Line Arguments

The Naughty Dog engine scans the command line for a predefined set of special options. The name of the level to load can be specified, along with a number of other commonly used arguments.

Scheme Data Definitions

The vast majority of engine and game configuration information in the Naughty Dog engine (used to produce the *Uncharted* and *The Last of Us* series) is specified using a Lisp-like language called Scheme. Using a proprietary data compiler, data structures defined in the Scheme language are transformed into binary files that can be loaded by the engine. The data compiler also spits out header files containing C `struct` declarations for every data type defined in Scheme. These header files allow the engine to properly interpret the data contained in the loaded binary files. The binary files can even be recompiled and reloaded on the fly, allowing developers to alter the data in Scheme and see the effects of their changes immediately (as long as data members are not added or removed, as that would require a recompile of the engine).

The following example illustrates the creation of a data structure specifying the properties of an animation. It then exports three unique animations to the game. You may have never read Scheme code before, but for this relatively simple example it should be pretty self-explanatory. One oddity you'll notice is that hyphens are permitted within Scheme symbols, so `simple-animation` is a single symbol (unlike in C/C++ where `simple-animation` would be the subtraction of two variables, `simple` and `animation`).

simple-animation.scm

```
;; Define a new data type called simple-animation.
(deftype simple-animation ()
    (
        (name              string)
        (speed             float   :default 1.0)
```

```
        (fade-in-seconds  float   :default 0.25)
        (fade-out-seconds float   :default 0.25)
    )
)

;; Now define three instances of this data structure...
(define-export anim-walk
    (new simple-animation
        :name "walk"
        :speed 1.0
    )
)

(define-export anim-walk-fast
    (new simple-animation
        :name "walk"
        :speed 2.0
    )
)

(define-export anim-jump
    (new simple-animation
        :name "jump"
        :fade-in-seconds 0.1
        :fade-out-seconds 0.1
    )
)
```

This Scheme code would generate the following C/C++ header file:

simple-animation.h

```
// WARNING: This file was automatically generated from
// Scheme. Do not hand-edit.

struct SimpleAnimation
{
    const char* m_name;
    float       m_speed;
    float       m_fadeInSeconds;
    float       m_fadeOutSeconds;
};
```

In-game, the data can be read by calling the `LookupSymbol()` function, which is templated on the data type returned, as follows:

```
#include "simple-animation.h"
void someFunction()
{
    SimpleAnimation* pWalkAnim
        = LookupSymbol<SimpleAnimation*>(
            SID("anim-walk"));

    SimpleAnimation* pFastWalkAnim
        = LookupSymbol<SimpleAnimation*>(
            SID("anim-walk-fast"));

    SimpleAnimation* pJumpAnim
        = LookupSymbol<SimpleAnimation*>(
            SID("anim-jump"));

    // use the data here...
}
```

This system gives the programmers a great deal of flexibility in defining all sorts of configuration data—from simple Boolean, floating-point and string options all the way to complex, nested, interconnected data structures. It is used to specify detailed animation trees, physics parameters, player mechanics and so on.

7
Resources and the File System

Games are by nature multimedia experiences. A game engine therefore needs to be capable of loading and managing a wide variety of different kinds of media—texture bitmaps, 3D mesh data, animations, audio clips, collision and physics data, game world layouts, and the list goes on. Moreover, because memory is usually scarce, a game engine needs to ensure that only one copy of each media file is loaded into memory at any given time. For example, if five meshes share the same texture, then we would like to have only one copy of that texture in memory, not five. Most game engines employ some kind of *resource manager* (a.k.a. *asset manager*, a.k.a. *media manager*) to load and manage the myriad resources that make up a modern 3D game.

Every resource manager makes heavy use of the file system. On a personal computer, the file system is exposed to the programmer via a library of operating system calls. However, game engines often "wrap" the native file system API in an engine-specific API, for two primary reasons. First, the engine might be cross-platform, in which case the game engine's file system API can shield the rest of the software from differences between different target hardware platforms. Second, the operating system's file system API might not provide all the tools needed by a game engine. For example, many engines support file *streaming* (i.e., the ability to load data "on the fly" while the game is running), yet most operating systems don't provide a streaming file system API